DSA Answer Scripts for SWE1 Interviews

Rohit Jangir

1. Sliding Window

Q: Explain the Sliding Window Technique.

Answer: Sliding Window is used for problems involving contiguous subarrays or substrings. We maintain a window defined by two pointers (left and right), expand the right pointer to grow the window, and shrink from the left when constraints are violated. This converts brute-force $O(n^2)$ solutions into optimal $O(n)$ solutions, commonly used for substring problems like longest substring without repetition or fixed-size subarray sums.

2. Two-Pointers

Q: When do you use the Two-Pointer technique?

Answer: Two pointers are used when the input is sorted or when we process elements from both ends simultaneously. Typical examples include pair-sum in sorted arrays, removing duplicates, merging arrays, or reversing sequences. It improves efficiency by avoiding nested loops.

3. Prefix Sum + HashMap

Q: How do Prefix Sums help in subarray problems?

Answer: Prefix sums turn range-sum problems into constant-time queries. If prefix sum up to index i is S_i, then sum of subarray $(l..r)$ is $S_r - S_{l-1}$. Using a HashMap, we can count how many previous prefix sums match a target condition. This allows $O(n)$ solutions for subarray sum equals k.

4. Binary Search

Q: Why is Binary Search $O(\log n)$?

Answer: Binary search repeatedly halves the search space. Each iteration removes half of the remaining elements, creating a logarithmic time complexity. It is commonly used in sorted arrays, rotated sorted arrays, and "binary search on answer" problems.

5. Divide and Conquer

Q: Explain Merge Sort.

Answer: Merge Sort divides the array into halves, recursively sorts each half, and merges them back. The merge step is linear, and there are $\log n$ levels of division, giving a total time of $O(n \log n)$. It is stable and predictable in performance.

6. Dynamic Programming

Q: What is Dynamic Programming?

Answer: DP solves problems with overlapping subproblems by storing results of smaller subproblems and reusing them instead of recomputing. Two approaches are used: memoization (top-down) and tabulation (bottom-up). Examples include Fibonacci, coin change, knapsack, and longest increasing subsequence.

7. Greedy Algorithms

Q: When is Greedy applicable?

Answer: Greedy algorithms make the best local choice at each step with the hope of reaching a global optimum. They work when the problem exhibits the greedy-choice property and optimal substructure. Examples include interval scheduling, minimum arrows, and activity selection.

8. Tree Traversals

Q: Explain DFS and BFS on Trees.

Answer: DFS explores deep first using recursion or an explicit stack; used for recursion-heavy problems such as diameter, validation, and height. BFS explores level by level using a queue; used for shortest paths and level-order traversals.

9. Graph Traversals

Q: What is the difference between BFS and DFS in graphs?

Answer: BFS explores neighbors first and is ideal for shortest paths in unweighted graphs. DFS explores depth-first and is useful for cycle detection, topological sort, and connected components.

10. Heaps and Priority Queues

Q: What is the internal structure of a Priority Queue?

Answer: Java's PriorityQueue is implemented as a binary heap. Insertions and deletions take $O(\log n)$ time, and the smallest element (in a min-heap) is always accessible in $O(1)$ time.